

DNS Reconnaissance **Mastery**

From "Why DNS Matters" to Zone Transfers and the Complete Recon Methodology — Lesson 1 to the Final Lesson

Prepared for: Muhammad Shafiq Goraya

Track: Red Team / Penetration Testing

Style: Plain explanations, real-world examples, hands-on practicals, no quiz format

Covers: Why penetration testers start with DNS, hands-on tool demonstrations (nslookup, dig, host, reverse DNS, MX lookups), Zone Transfer (AXFR) fundamentals, reading AXFR results like a professional, and the complete passive/active reconnaissance methodology

DNS

nslookup

dig

host

Reverse DNS

MX Records

Zone Transfer

AXFR

Attack Surface Mapping

Recon Pyramid

WHAT'S INSIDE

- Lesson 1 — Why Does a Penetration Tester Start With DNS?
- Foundational Practical Walkthrough — nslookup, dig, host, Reverse DNS, MX Records
- Lesson 9 — Zone Transfer (AXFR): What It Is and Why It Exists
- Lesson 10 — Reading an AXFR Like a Pentester
- Lesson 11 — The Complete Information Gathering Methodology
- Practical Lab Exercises — Hands-On DNS Recon
- Final Summary + Keyword Grid + Homework

Lesson 1 · Why Does a Penetration Tester Start With DNS?

Think Like a Thief

Imagine This

You are standing outside a huge company building. You are not allowed to enter yet. What do you do first? Do you break the door? No — a smart thief first observes: How many doors are there? How many cameras? Which employees come in? Which entrance is used? Where is security weak? This is called Reconnaissance (Recon). A penetration tester does exactly the same thing — except instead of a physical building, we look at the company's digital building.

What is DNS?

DNS stands for Domain Name System. It is basically the Internet's phonebook. Humans remember names like google.com or tesla.com, but computers actually communicate using numeric addresses like 142.250.183.46. DNS converts a name like google.com into the IP address that computers actually use to talk to each other.

Why Does an Ethical Hacker Care?

This is where beginners and professionals think differently. A beginner thinks "DNS only tells me the IP address." A professional thinks "DNS leaks information." Information is power — sometimes more valuable than a vulnerability itself.

Example

Suppose a company hires you and says: "Our website is abccompany.com." Most beginners immediately run `nmap abccompany.com`. A senior pentester says "Wait" and first asks: What else belongs to this company? How many servers exist? Are they using AWS? Are they using Microsoft? Is mail hosted somewhere? Are there hidden subdomains? Which IPs belong to them? DNS helps answer all of these questions before a single scan is even run.

Think Like Sherlock Holmes

Imagine you only know one thing: the target is tesla.com. From DNS alone you might discover mail.tesla.com, vpn.tesla.com, dev.tesla.com, api.tesla.com, and shop.tesla.com. Now instead of one target, you suddenly have five. Every new asset could contain its own vulnerability. This is exactly why bug bounty hunters spend hours doing reconnaissance before ever touching Burp Suite or Metasploit.

The Golden Rule

The more you know about the target, the easier it becomes to hack it. Recon saves time. Recon finds forgotten systems. Recon finds weak systems. Recon often finds the vulnerability itself.

Real Bug Bounty Example

Example

Imagine a company protects `www.company.com` very well. But DNS reveals `old.company.com` — a server nobody remembered, which hasn't been updated in five years and still runs an old version of WordPress. The hacker compromises that forgotten server instead. Notice: the vulnerability wasn't found by scanning — it was found by good reconnaissance.

Key Takeaways

- DNS is much more than name-to-IP conversion.
- DNS is one of the very first reconnaissance sources a pentester uses.
- A pentester uses DNS to discover assets, not just resolve one address.
- More assets means more attack surface.
- Good recon often leads directly to successful exploitation later.

Something to Sit With

Suppose a company only gives you one thing: `example.com`. Why shouldn't you immediately start scanning `example.com`? What should you try to learn first using DNS? Keep this question in mind as you move through the rest of this document — the answer becomes clear by the end.

Foundational Practical Walkthrough · `nslookup`, `dig`, `host`, Reverse DNS & MX Records

Note: the original source material jumps from Lesson 1 directly into a live tool demonstration, then later resumes at Lesson 9. The lessons in between (2 through 8) were not included in the material provided, so this section presents the live demonstration content in full, bridging Lesson 1 and Lesson 9.

Why DNS Verification Matters First

Before a penetration tester touches any system, one of the most fundamental reasons to do DNS recon is verification. When a client hires a penetration testing company, they hand over some IPs or domain names and say "test this." But people are imperfect — they can give the wrong IP addresses by mistake. Even more importantly, that person may not actually have the authorization to grant permission to test systems they don't own.

Key Idea

DNS is used for verification — confirming that the systems a client claims to own are actually a real, traceable part of their organization. Since DNS is a global system, not something any one company privately controls, you can independently trace an IP address back to an official registrar to confirm ownership, rather than blindly trusting what you were told.

Basic Name Resolution — `nslookup`

`nslookup` is a classic, widely available tool for DNS resolution.

```
nslookup itpro.tv
```

Running this returns a "non-authoritative answer" containing one or more IP addresses. Seeing multiple IPs for one domain often means load balancing is happening, or the site is going through a CDN (Content Delivery Network) — it's not simply a 1-to-1 domain-to-server relationship.

Why This Matters for Scope

If a client says "anything under itpro.tv is in scope," and DNS resolution reveals four different IP addresses tied to that domain, you now have four in-scope systems to add to your target list — not just one. This is especially important in a black box test, where you're looking at the organization from the outside in, exactly like a real attacker would.

Alternative Tools — dig and host

On Linux systems, dig and host are extremely common alternatives to nslookup, usually available by default or installable via your package manager (apt-get or yum).

```
dig itpro.tv
```

The output format looks different from nslookup, but the important part is the answer section — this is where you'll find the same IP addresses, labeled as A records.

```
host itpro.tv
```

Running host also returns the IP addresses, but often includes extra detail. For example, it might reveal that mail is handled by itpro-tv.mail.protection.outlook.com — telling you the organization uses Outlook for email, and that some form of mail protection service is in place.

Thinking Like an Attacker

Learning that a target uses Outlook for mail isn't just trivia — it opens up questions like "could a social engineering attack targeting Outlook users be useful here?" DNS often gives you more than you initially bargained for.

Finding Primary and Secondary Name Servers

To find out who actually manages DNS for an organization — the "main players" — use the -t flag with host to specify the record type:

```
host -t ns itpro.tv
```

This might return something like ns-1.awsdn-00.com, revealing that the organization is using a cloud service (in this case, AWS) to provide its DNS. This is more free, publicly available information that helps you understand the target's infrastructure — nothing here involves bypassing any security control.

Reverse DNS Lookups

Sometimes you'll only be given an IP address, with no domain name attached. In that case, you can perform a reverse DNS lookup to see if that IP resolves back to a hostname.

```
dig -x [IP address]
```

The -x flag tells dig to perform a reverse lookup instead of a normal forward lookup. The result can sometimes be surprising — you might expect to see the domain name you started with, but instead get something like

102.31.230.54.in-addr.arpa , which is the special reverse-lookup domain format DNS uses internally. This is still useful — it's all part of painting a fuller picture of the underlying infrastructure and how systems are connected.

Checking Mail Exchange (MX) Records

To specifically check how a domain's email is handled, use the `-t` flag with the `mx` type:

```
dig -t mx itpro.tv
```

This returns just the mail exchange record for the domain — for example, confirming again that mail is routed through `itpro-tv.mail.protection.outlook.com`. This is a fast, targeted way to learn about mail infrastructure specifically, without wading through unrelated DNS records.

A Live Zone Transfer Demonstration

Zone transfers come up when discussing DNS misconfigurations. A zone transfer allows a secondary DNS server to copy the entire DNS database from the primary — necessary for legitimate redundancy, but dangerous if any outsider can request the same copy. Most organizations (including `itpro.tv`, in this demonstration) do not allow full zone transfers anymore, because it's a well-understood security risk.

To practice this safely, a purpose-built practice domain is used instead:

```
host -t ns zonetransfer.me
```

This reveals the name servers for `zonetransfer.me`. Once you have a name server, you can attempt the transfer itself:

```
dig axfr zonetransfer.me @[name server]
```

Running this against a domain that allows it returns the entire DNS database for that name server — every record, every subdomain, all at once. In the demonstration, this revealed interesting entries like `deadbeef.zonetransfer.me`, along with many other subdomains and record types. This is the clearest possible illustration of why a successful zone transfer is so valuable to an attacker — it turns slow, one-at-a-time discovery into an instant, complete map.

Bridge to the Next Lessons

This live demonstration is exactly what Lessons 9 and 10 (below) unpack in much greater depth — why zone transfers exist, why they're dangerous when misconfigured, and how to actually read and prioritize a large AXFR result like a professional instead of just being impressed by the wall of text.

Lesson 9 · Zone Transfer (AXFR) — What It Is and Why It Exists

This is one of the most valuable techniques you'll use during DNS reconnaissance. Before memorizing the command, it's important to understand why Zone Transfers exist in the first place.

Why Companies Need a Backup DNS Server

Imagine a Company

A company has one Primary DNS server holding all its records: `www.company.com`, `mail.company.com`, `vpn.company.com`, and so on. Now imagine that Primary DNS server crashes. Nobody can resolve the company's domain — the website disappears, emails stop working, and VPN users can't connect. This is completely unacceptable for a real business.

The solution is to run a Secondary DNS server as a backup. But this raises a question: how does the Secondary DNS server get a copy of every record from the Primary? It asks the Primary DNS: "Please send me the whole DNS database." That copying process is called a **Zone Transfer**, technically known as **AXFR**.

The Administrator's View vs the Attacker's View

Normal Administrator's View

Primary DNS copies its database to the Secondary DNS. Everything is normal, expected, and intentional — this is exactly how DNS redundancy is supposed to work.

Attacker's View

Now imagine an attacker pretends to be a secondary DNS server and asks the Primary DNS: "Can I have the entire DNS database?" If the DNS server is misconfigured, it replies "Sure" — and the attacker instantly receives every hostname: `www`, `mail`, `vpn`, `git`, `dev`, `backup`, `db`, `ftp`, and more. Instead of discovering one hostname at a time, the attacker gets the entire map in one shot.

Why This Is So Dangerous

Remember that DNS recon tries to answer one core question: what assets exist? Normally, you discover assets slowly, one lookup at a time. A successful AXFR changes everything — one single command can reveal hundreds of records instantly, collapsing what would normally take hours of slow enumeration into seconds.

Why Most Companies Disable It

A properly configured DNS server checks who is asking before responding: "Who are you? You're not my Secondary DNS. Denied." Only authorized secondary DNS servers should ever be allowed to perform an AXFR against the primary.

Practical Example

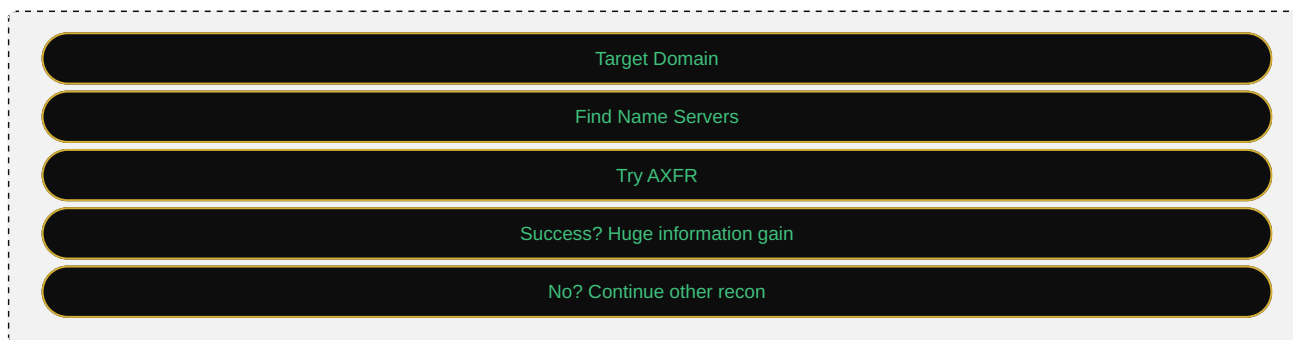
```
dig axfr zonetransfer.me @nsztml.digi.ninja
```

This works because `zonetransfer.me` is intentionally configured to allow AXFR, specifically so students can practice this technique safely and legally. If you try the same command against a properly secured domain:

```
dig axfr google.com
```

You'll almost certainly get "Transfer failed" or "Transfer refused." That refusal is the expected and secure behavior — it means the DNS server is configured correctly.

Pentester Methodology for AXFR



Important Distinction

Trying an AXFR is normal enumeration, not exploitation. You are not exploiting a vulnerability in software — you're simply checking whether the DNS server has a misconfiguration. This is a legitimate, standard part of authorized reconnaissance.

Lesson 10 · Reading an AXFR Like a Pentester

Most beginners stop at "Wow, lots of records!" A senior pentester instead asks: "Which of these records actually matter?" Simply running the command isn't enough — knowing how to read the output is what separates a beginner from a professional.

```
dig axfr zonetransfer.me @nsztl1.digi.ninja
```

You'll see many lines like `www.zonetransfer.me`, `mail.zonetransfer.me`, `ftp.zonetransfer.me`, `sip.zonetransfer.me`, `staging.zonetransfer.me`, and more. At first, it looks overwhelming. Don't read it from top to bottom — categorize it instead.

Step 1 — Look for Interesting Hostnames

Not every hostname deserves equal attention. Imagine you see `www.company.com` — is this interesting? Not really; you already expected a website to exist. But now you see `staging.company.com` — this should immediately wake your brain up.

Why Staging Environments Matter

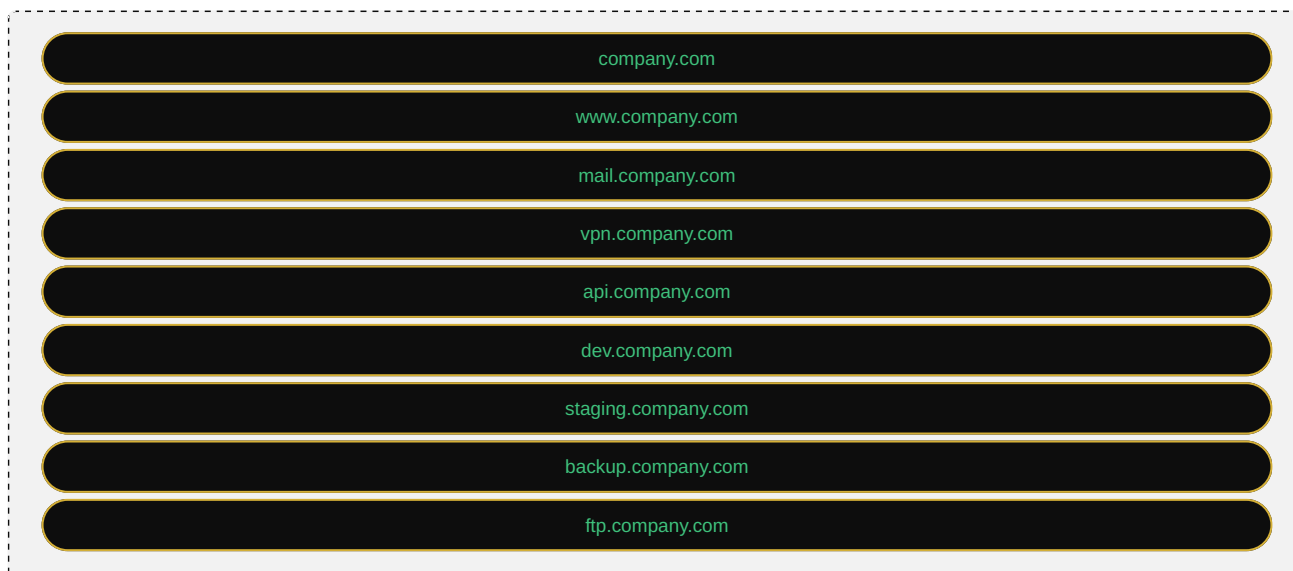
Staging environments often run older code, have weaker security, and are forgotten by administrators once the "real" production system is live. This doesn't automatically mean it's vulnerable — it means it's now a high-priority target for further investigation.

Another example is `dev.company.com`. The right questions to ask are: Is it Internet-accessible? Is debugging enabled? Is it using test credentials? Is it actually in scope for this engagement?

One more example is `backup.company.com`. This is interesting because it may expose backup files, may have weaker authentication, and may reveal sensitive information that was never meant to be public-facing. Again — you're prioritizing what to look at next, not immediately exploiting anything.

Step 2 — Build an Attack Surface Map

Instead of seeing a flat list of names, organize what you found mentally, like a tree:



Organized this way, you now understand the company's Internet-facing assets much more clearly than a flat, unstructured list would show.

Step 3 — Prioritize

Not every hostname deserves the same attention. A senior pentester ranks discovered assets by risk and value, not by curiosity.

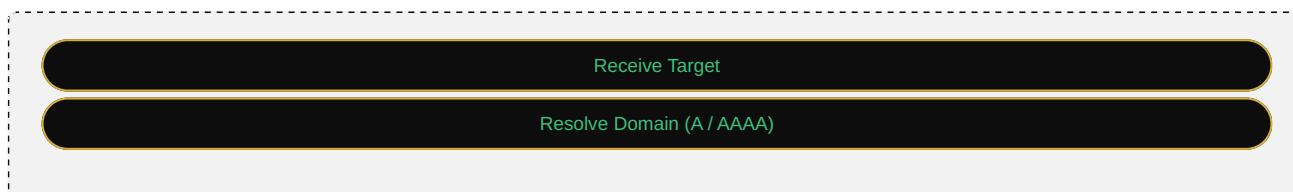
Host	Priority	Why
vpn.company.com	High	Remote access point
staging.company.com	High	Often less secure
dev.company.com	High	Development mistakes happen
backup.company.com	Medium–High	May expose sensitive data
mail.company.com	Medium	Mail infrastructure
www.company.com	Medium	Usually already well protected

Real-World Scenario

Example

Suppose an AXFR reveals `jenkins.company.com`. You don't immediately try passwords against it. Instead you ask: Is Jenkins publicly accessible? Which version is it running? Does it require authentication? Is it actually in scope? You're gathering evidence and context before taking any action.

The Complete DNS Methodology Built So Far



Check MX Records

Check NS Records

Identify DNS Provider

Attempt AXFR (if appropriate)

If Successful: Build Attack Surface + Prioritize

Begin Service Enumeration

Notice that Nmap hasn't even appeared yet — that's intentional. A professional gathers intelligence first, then scans with purpose.

The Biggest Lesson From This Lesson

Reconnaissance is about answering questions, not collecting commands. Every command should answer a specific question: `host company.com` asks "Where is it?" `host -t mx company.com` asks "How is email handled?" `host -t ns company.com` asks "Who manages DNS?" `dig axfr ...` asks "Can I discover the entire DNS zone?" If you remember this, you stop being someone who memorizes tools and become someone who understands why they're using them.

Lesson 11 · The Complete Information Gathering Methodology

DNS is only one piece of reconnaissance. In a real penetration test, DNS is one box in a much larger workflow. Imagine a client gives you a single target: `company.com`. Your brain should automatically build out a much bigger picture from there.

`company.com`

Passive Recon (No Direct Interaction)

Active Recon (Direct Interaction)

Phase 1 — Passive Recon

Passive means collecting information without touching the target's servers directly. Examples include Google searches, DNS records, WHOIS lookups, Certificate Transparency logs, GitHub, LinkedIn, Shodan, and Archive.org. Think of it like being a detective reading public records — no alarms, no logs on the target's side, no scanning.

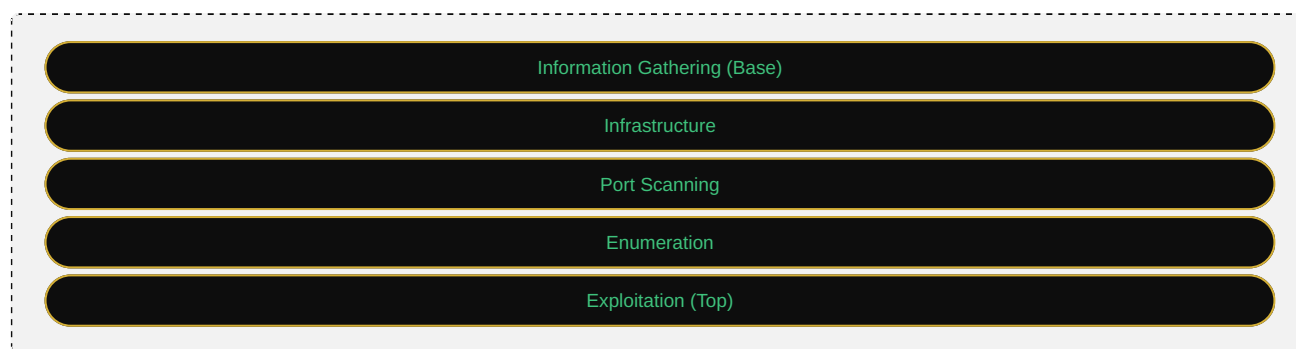
Phase 2 — Active Recon

Now you start directly interacting with the target — for example running `host company.com`, `dig company.com`, or `nslookup company.com`, and later tools like Nmap. From this point forward, the company's systems can potentially see your requests.

The Entire Attack Surface

DNS helped find one website. But in reality, a company's full footprint looks much bigger — a website, a mail server, a VPN, and under the website itself there might be an API, a dev environment, a staging environment, and even an admin portal. Your job isn't to attack everything immediately. Your job is to find everything first.

The Recon Pyramid



Every layer depends on the one below it. Beginners often skip straight to the top. Professionals build methodically from the bottom up.

Real Bug Bounty Example

Two Hackers, Same Target

Hacker A immediately runs Nmap against company.com, finds nothing interesting, and leaves. Hacker B spends 30 minutes on proper reconnaissance first, and discovers staging.company.com — an old application, running outdated software, with a known vulnerability — resulting in a valid, paid bug bounty submission. Same target, completely different methodology, completely different outcome.

A Practical Recon Checklist

1. Who owns it?

WHOIS records, ASN (Autonomous System Number), and the organization behind the domain.

2. Where is it hosted?

AWS? Azure? Cloudflare? Google Cloud? Self-hosted infrastructure? This shapes what kind of misconfigurations or third-party issues might apply.

3. What domains exist?

www, mail, vpn, api, staging, dev, admin — building the same kind of attack surface map covered in Lesson 10.

4. What technologies are used?

Apache, Nginx, IIS, Exchange, Jenkins, GitLab — knowing the technology stack narrows down what vulnerabilities are even worth checking for.

5. What services are exposed?

Only after all of the above is understood does real scanning begin — with purpose, not blindly.

The Biggest Lesson From This Entire Module

Reconnaissance reduces uncertainty. The less uncertainty you have, the fewer unnecessary scans you run, the fewer false assumptions you make, and the better your exploitation decisions become later in the engagement.

Practical Lab Exercises · Hands-On DNS Recon

Reading about these techniques builds understanding, but running them yourself builds real skill. All exercises below use safe, intentionally-public practice targets — never attempt AXFR or active recon against a domain you don't have explicit permission to test.

Lab 1 — Basic Resolution With Three Tools

Pick any domain you're comfortable testing (or a practice domain) and run all three:

```
nslookup example.com
dig example.com
host example.com
```

Compare the three outputs. Note what information each tool shows that the others don't.

Lab 2 — Find the Name Servers

Run:

```
host -t ns zonetransfer.me
```

This returns the authoritative name servers. Before moving to the next lab, write down which name server you would target and why.

Lab 3 — Perform the Zone Transfer

Run:

```
dig axfr zonetransfer.me @nsztl1.digi.ninja
```

Review the full output. Categorize each hostname the way Lesson 10 described: expected (like www) versus interesting (like staging, dev, backup, or an unfamiliar service name).

Lab 4 — Reverse DNS and MX Practice

Pick an IP address you already resolved in Lab 1, then run:

```
dig -x [that IP address]
dig -t mx example.com
```

Note whether the reverse lookup returns the domain you expected, or something else — and record what mail provider the MX lookup reveals.

Lab 5 — Build Your Own Attack Surface Map

Using the results from Lab 3, draw out a simple tree diagram (on paper or in a text file) showing zonetransfer.me at the top, with every discovered subdomain branching underneath it — exactly like the tree structure shown in Lesson 10.

Lab 6 — Prioritize Your Findings

Create a simple three-column table — Host, Priority, Reason — and rank every hostname you found from Lab 3 using the same High / Medium-High / Medium logic shown in Lesson 10's priority table. Justify each ranking in your own words.

Lab 7 — Test the Negative Case

Run:

```
dig axfr google.com
```

Observe the "Transfer failed" or "Transfer refused" response. Write one or two sentences explaining, in your own words, why this refusal is actually the correct and secure behavior.

Lab 8 — Full Recon Checklist Practice

Pick any domain you are authorized to test (or a deliberately vulnerable practice target from TryHackMe or Hack The Box), and manually work through the 5-point recon checklist from Lesson 11: who owns it, where is it hosted, what domains exist, what technologies are used, and only then, what services are exposed. Do not run Nmap until you've completed steps 1 through 4 on paper first.

Reminder

Every one of these labs is passive or explicitly-authorized active enumeration. Never run AXFR, Nmap, or any active technique against infrastructure you don't own or have written permission to test.

Final Summary · What You've Learned in This Module

- Why penetration testers start with DNS, and why it's used for verification, not just resolution
- What DNS is and why pentesters care about it
- How to use nslookup, dig, and host for basic resolution, name server lookups, reverse DNS, and MX records
- A, AAAA, MX, and NS records
- Non-authoritative vs authoritative answers
- Why one domain can have multiple IPs
- How MX records reveal mail infrastructure, and NS records reveal DNS providers
- Why cloud provider information matters
- What AXFR (Zone Transfer) is, and why it's dangerous when misconfigured
- How to read an AXFR result like a pentester — categorizing, mapping, and prioritizing rather than panicking at a wall of text
- The full passive and active reconnaissance methodology, and the Recon Pyramid
- How to think like a recon specialist instead of jumping straight to exploitation

The Full Keyword List

DNS	nslookup
dig	host
Reverse DNS	MX Record
Zone Transfer	AXFR
Attack Surface	Passive Recon
Active Recon	Recon Pyramid

Homework (15 minutes)

Without looking at your notes, explain these topics in simple English, in your own words:

- Why DNS is used for verification before a penetration test even begins
- The difference between what nslookup, dig, and host each show you
- Why Zone Transfers exist in the first place, and why they can be dangerous
- Why "Transfer refused" is actually a good outcome, not a failure
- The difference between discovering a hostname and prioritizing a hostname
- The difference between passive and active reconnaissance
- Why "reconnaissance reduces uncertainty" is the single most important lesson from this module

If you can teach these topics to someone else in simple English, you have truly understood them. Get some rest — you've earned it.